ADVANCED AI SOCIETY · PROOF-OF-CONTROL INITIATIVE
Working draft v0.1 · August 2026 · for co-author review prior to submission

Proof-of-Control: An Open Standard for Runtime Verifiability and Cryptographic Oversight in Autonomous AI Execution

— Prove, rather than promise, what your agents did.

Jim Schwoebel[1,2], Ken Huang[2], Tricia Wang[2], Michael Casey[2], Sheila Warren[2], Julie Tsai[2], Bettina Warburg[2], Noah Ringler[2], Anni Lai[2], Paul Calatayud[2], Ahmer Inam[2], Brian Behlendorf[2], Clay Shirky[2], Sunny Bates[2], Cyrus Hodes[2], Jana Eggers[2], Nicolaj Waldorf[2], Addie Wagenknecht[2], Paul Brody[2], Andrew Nebus[2], Irina Marinescu[2], Esteban Kolsky[2], Ray Wang[2], James Andrews[2], Benjamin Palmer[2], David Bray[2], John Kuch[2], Nelson Rosario[2], Gold Darr[2], Ben Christiansen[2], Sam Nathans[2], Heather Lord[2]

[1] Quome [2] Advanced AI Society, `https://advancedaisociety.org`[1]

ABSTRACT

Autonomous AI agents plan multi-step workflows, invoke external APIs, modify application state, and delegate sub-tasks with minimal real-time human supervision—exposing a systemic vulnerability we call the *Verifiability Gap*: the structural impossibility of confirming that an agent operated within its authorized policy boundaries and that its controls remained active and unmanipulated. Legacy mechanisms (system prompts, static access control, embedded guardrails, post-hoc log auditing) fail against non-deterministic, path-dependent execution at machine speed. We introduce Proof-of-Control (PoC), an open standard for runtime verifiability comprising: (i) six *domains of verification* enumerating 111 auditable requirements; (ii) four *Verifiability Tiers* grading evidence by who must be trusted to believe it, with a *binary threshold* separating authenticated documentation from mechanism-generated evidence; (iii) a reference architecture coupling lifecycle interception hooks with hardware-isolated policy evaluation under IETF RATS (RFC 9334), emitting Entity Attestation Tokens over a signed Merkle execution chain; and (iv) a reproducible coverage mapping showing that eight leading frameworks (NIST AI RMF, ISO/IEC 42001, OWASP AISVS, EU AI Act, SOC 2, CSA AARM, NIST SP 800-207, MITRE ATLAS) address 27–64% of PoC requirements—almost entirely as partial matches that require the control but not operator-independent evidence of it. We state a formal policy model, a zero-trust threat model, a sub-15 ms latency budget, and an integration roadmap across ISO/IEC JTC 1/SC 42, IETF RATS, and NIST AI 600-1.

[1]Draft prepared within the Proof-of-Control Initiative of the Advanced AI Society. The author list reflects the Initiative's leadership, board, and advisory board; individual consent to co-authorship is being collected before submission, and the list will be revised to include only consenting contributors.

1 Introduction

The rapid evolution from passive natural-language generation to autonomous agentic execution represents a foundational shift in computational infrastructure [1, 2]. Autonomous agents now plan multi-step operational workflows, query complex databases, invoke external APIs, modify application state, and delegate sub-tasks to secondary agents with minimal real-time human supervision. While this autonomy unlocks substantial productivity, it exposes a systemic architectural vulnerability: the *Verifiability Gap*—the absence of independent evidence of what an AI system did [3].

The Verifiability Gap describes the structural impossibility of confirming—either during execution or retrospectively—that an autonomous agent operated within its authorized policy boundaries, that mandatory security controls remained active and uncompromised, and that the agent's state transitions were free from malicious manipulation or reasoning failure. Because agents execute complex, non-deterministic actions at machine speed, human oversight and traditional log-auditing workflows cannot maintain effective operational control [4]. Existing governance implementations rely almost exclusively on operator self-attestation, static permissions, or unanchored application logs, none of which provide open, cryptographically verifiable proof of policy compliance [5].

Contributions. This paper makes five contributions: (1) we define the Proof-of-Control standard—six domains of verification, four evidence properties, four Verifiability Tiers with a binary threshold, four requirement levels aligned to the tiers, and three conformance stages—as a catalogue of 111 auditable requirements (Section 4); (2) we specify a reference architecture integrating lifecycle interception with hardware-backed attestation under IETF RATS [6], Entity Attestation Tokens [7], and signed Merkle execution chains [8] (Section 5); (3) we formalize the policy model and analyze security under a zero-trust threat model (Sections 6–7); (4) we report a reproducible, requirement-level coverage evaluation against eight regulatory and security frameworks using the coding methodology of HAARF [9], together with a latency budget (Section 8); and (5) we lay out an integration roadmap across ISO/IEC JTC 1/SC 42 [10, 11], the IETF RATS working group [12], the Agent Control Specification [13, 14], and NIST AI 600-1 [15] (Section 9).

2 Background and Motivation

2.1 Path dependency and non-deterministic execution

Unlike legacy software executing deterministic decision trees, agents operate within LLM reasoning loops that yield non-deterministic, path-dependent trajectories [5]. Given a single objective, an agent dynamically determines its intermediate steps, selects tools, interprets unstructured outputs, and modifies its path from real-time observations. Compliance therefore cannot be validated prior to deployment, and evaluating an individual tool call in isolation fails to mitigate path-dependent risk: an agent reading confidential records early and issuing an outbound network request later creates a cumulative exfiltration risk that single-step inspection cannot detect. Recent empirical work quantifies this: under skill composition, attack success on capability-flow paths rises from near zero in isolation to 33.6%, trust-transfer attacks succeed on more than 96.5% of attempts across four of five model backends, and contextual contamination raises risky approvals by 71.8% [16].

2.2 Why legacy governance fails

Current AI risk management relies on three primary control mechanisms, none of which resolves the Verifiability Gap.

System prompts and in-context instructions inhabit the same logical stream as untrusted user inputs, retrieved documents, and tool outputs; this co-mingling renders them vulnerable to direct and indirect prompt injection. Prompting is a probabilistic steering mechanism, not a deterministic boundary: if a model deviates from its system prompt, no execution barrier prevents the action [5].

Role-based access control and static identity restrict which endpoints an agent may call but operate without contextual awareness. An agent holding valid credentials for both internal database reads and external messaging retains the technical capability to exfiltrate data; static IAM cannot evaluate the context, sequence, or cumulative risk of individually authorized permissions [17].

Application-embedded guardrails run under the agent's own execution authority. For agents with code-generation or system-command tools, internal guardrails provide weak isolation—a compromised process can bypass its own application-level checks—and they generate local, unauthenticated logs that cannot be audited independently of the host operator [18].

2.3 The collapse of the post-hoc audit window

The operational window for an autonomous exploit has collapsed toward zero: indirect prompt injections embedded in public web pages or internal repositories can hijack a reasoning loop instantly [2]. Retrospective SIEM auditing is insufficient because application logs lack cryptographic immutability; an operator—or an adversary with persistence—can modify, append, or delete entries post-execution. Verification must therefore shift from reactive log auditing to inline, machine-speed cryptographic attestation anchored to hardware roots of trust [19, 20].

3 Related Work

Runtime governance for agents. The Agent Control Specification defines portable interception middleware, policy manifests, and semantic conventions for agent runtimes [13, 14]; recent work frames runtime governance as policies over execution paths [5], and CSA's AARM defines approve/modify/defer/deny enforcement at the action boundary [21]. These supply the *enforcement* half of agentic assurance; Proof-of-Control supplies the complementary *evidence* half—the operator-independent proof that enforcement occurred.

Remote attestation and confidential computing. The IETF RATS architecture [6, 22], Entity Attestation Tokens [7, 23], multi-verifier topologies [24], and hardware TEEs with verifiable-compute toolchains [19, 20] provide the mechanism families Proof-of-Control builds on. These prove properties of *platforms and artifacts*; Proof-of-Control extends them into a conformance regime for *agent behavior*.

Empirical agent-safety evaluation. TraceSafe-Bench shows that guardrail efficacy on corrupted tool-calling trajectories correlates with structural trace parsing ($\rho = 0.79$ with JSON-validation competence) and near zero with natural-language safety benchmarks [25]; SCR-Bench demonstrates that artifact-level skill vetting is structurally blind to composition risk [16]. Both findings are absorbed as normative requirements in Proof-of-Control (path-aware authorization; validator trace-parsing competence).

Telemetry-first compliance and identity. AI Trust OS replaces questionnaire attestation with continuous, machine-collected control assertions and shadow-AI discovery from observability streams [26]; enterprise KRI taxonomies bridge frameworks to runtime signals [27]. Decentralized

identity work—W3C DIDs and Verifiable Credentials [28, 29], N-party trust circles [30], and supply-chain service passports [31]—provides the identity substrate for C1/C5.

Standards and regulation. NIST AI RMF [32] and its generative-AI profile [15], ISO/IEC 42001 [33] and 23894 [34], SOC 2 [35], the EU AI Act [36], OWASP AISVS [37], MITRE ATLAS [38], and zero-trust architecture [17] govern adjacent layers; Section 8 quantifies exactly how much of Proof-of-Control each already covers, following the coverage-mapping methodology introduced by HAARF for healthcare agent regulation [9].

4 The Proof-of-Control Standard

Proof-of-Control is *open verification* that the controls governing an agent system are implemented and hold, graded by how independently that can be verified. For each control it claims, across six domains, a conformant implementation MUST produce evidence—generated by the enforcing mechanism at execution time—that the control held; place that evidence on the Verifiability Tiers; and disclose its residual trust assumptions. A system has Proof-of-Control when, and only when, its evidence reaches Tier 3 or Tier 4 [3]. Requirements language follows RFC 2119 [39].

4.1 Six domains of verification

The standard enumerates 111 requirements across six domains, each stated as a testable *verify-that* obligation with auditor evidence guidance:

1. **Provenance** (C1): which model ran; lineage, artifact, and supply-chain origin, bound by hash-linked custody chains and signed manifests [40, 41, 31].
2. **Privacy** (C2): what data was read and written, evidenced without re-leaking the data being protected [36].
3. **Portability** (C3): boundary crossings—organizational, jurisdictional, compute—with signed linking records so the evidence chain survives migrations.
4. **Authorization** (C4): authority granted, decisions within or against it, delegation validity; *path-aware* evaluation so a composed sequence of individually authorized calls cannot exceed the authority of its steps [16].
5. **Identity** (C5): which agent and which principal ran, bound by short-lived delegation tokens, DIDs, and verifiable credentials [28, 29, 30].
6. **Security** (C6): integrity of the execution environment, controls held, tools invoked, key lifecycle in hardware-backed custody.

Four cross-cutting chapters govern evidence generation and properties (binary, contemporaneous, tamper-evident, transparent), tier grading, system-surface location on the seven MAESTRO layers [42], and conformance with standardized trust-assumption disclosure.

4.2 Verifiability Tiers and the binary threshold

Evidence is graded by *who must be trusted to believe it* (Figure 1). Cryptography alone does not raise the tier; removing the trusted party does. An operator-signed log is cryptographic and still Tier 2. A vendor-rooted TEE attestation is Tier 2 unless composed with independent anchoring

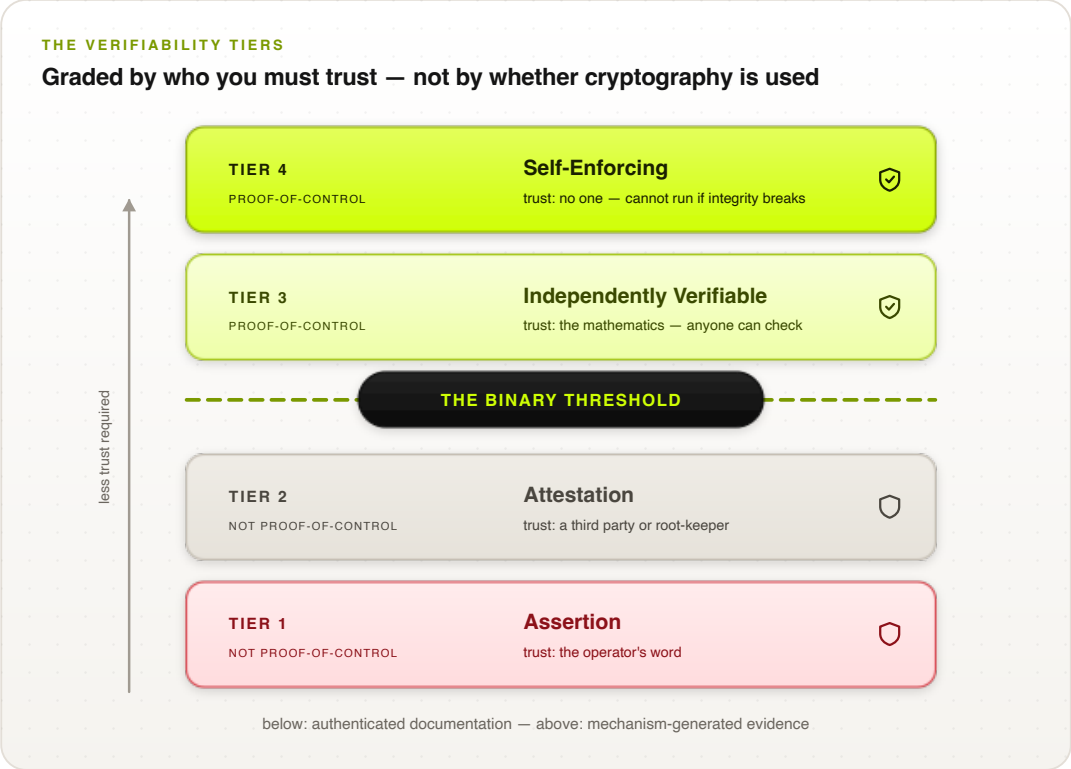

Figure 1: The four Verifiability Tiers. The binary threshold falls between Tiers 2 and 3: below it, authenticated documentation (a party must still be trusted); above it, mechanism-generated evidence. Requirement levels 1–4 align one-to-one with the tiers; meeting all Level 1–3 requirements in the claimed domains is the minimum for a Proof-of-Control claim.

(e.g., a public transparency log). The *binary threshold* falls between Tier 2 and Tier 3: below it, authenticated documentation; above it, mechanism-generated evidence. The threshold makes the category procurable—"does your AI have Proof-of-Control?" is a yes-or-no question, the pattern used by every category-defining certification (FIPS 140, Common Criteria, PCI DSS, CSA STAR).

Conformance is graded on a separate axis by three named stages—Self-Declared, Third-Party Assessed, Continuously Monitored—each requiring a standardized conformance statement with declared system boundary, in-scope action classes, per-claim tiers, mechanisms, and trust-assumption disclosure. The declared inventory MUST be reconciled against automated discovery from observability streams so undeclared ("shadow") agents surface as findings [26].

5 Reference Architecture

The reference architecture integrates runtime lifecycle interception with hardware-backed attestation under the IETF RATS architecture (RFC 9334) [6, 22]. Figure 2 shows the end-to-end pipeline and its mapping onto RATS roles: every consequential action is intercepted, reduced to a canonical state snapshot, evaluated inside a hardware enclave that the operator cannot reach into, and released only after signed evidence exists; Figure 3 shows the same boundary from the evidence-generation perspective.

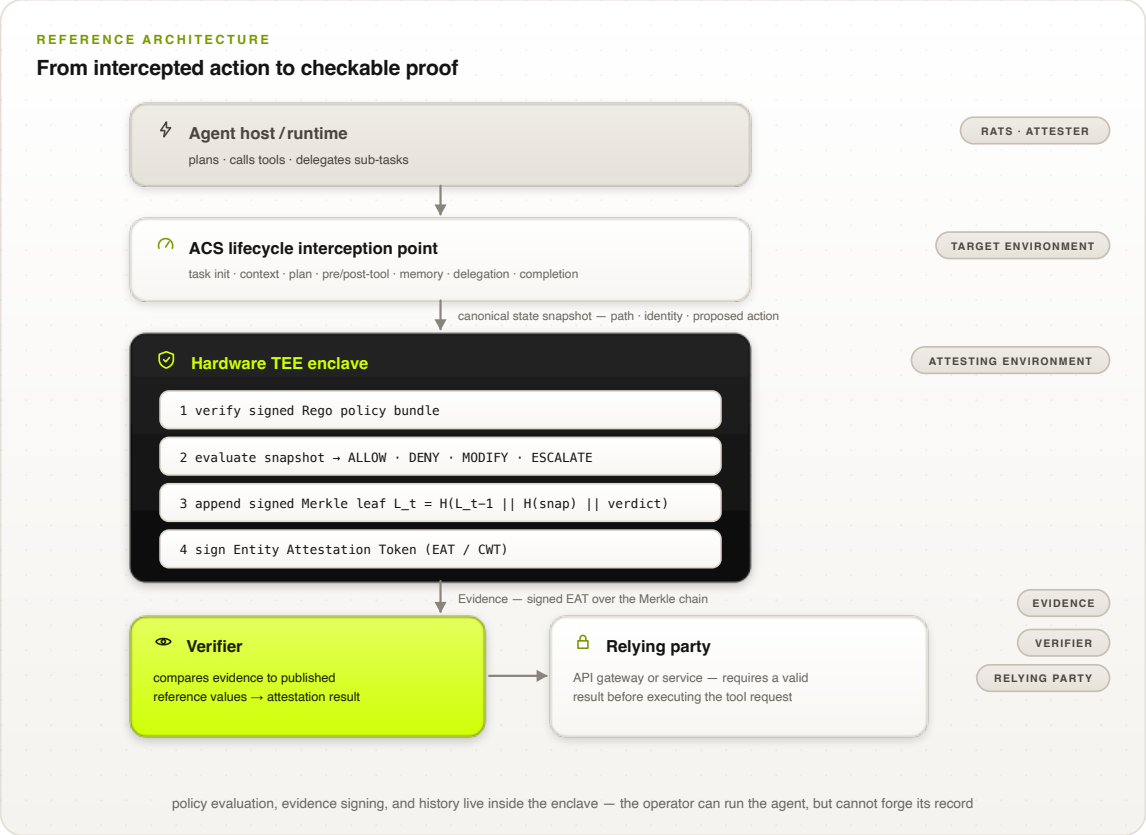

Figure 2: The Proof-of-Control reference architecture and its IETF RATS role mapping. The agent host (Attester) routes every lifecycle event through an ACS interception point; the hardware TEE enclave (Attesting Environment) verifies the policy bundle, evaluates the snapshot, extends the signed Merkle chain, and emits the Entity Attestation Token (Evidence), which a Verifier checks against published reference values before a Relying Party executes the request.

5.1 Lifecycle interception hooks

Proof-of-Control mandates standardized middleware hooks across the agent execution loop, leveraging the open Agent Control Specification (ACS) [13, 14]: *task initialization*, *context assembly* (RAG/memory injection), *plan generation*, *pre-call tool invocation*, *post-call tool result*, *memory write*, *sub-agent delegation* (with constraint inheritance), and *task completion*. At each point the host constructs a canonical state snapshot—agent identity, execution history, proposed action, environmental context—submitted to an isolated policy engine that returns a binding decision: ALLOW, DENY, MODIFY (inline sanitization), or ESCALATE (human-in-the-loop authorization). The gateway does not forward an action until the corresponding evidence record is durably written, and in-scope actions fail closed when evidence cannot be generated at the claimed tier.

5.2 RATS role mapping

The runtime maps onto RATS roles [6]: the *Attester* is the platform housing the agent runtime, hooks, and policy engine; the *Target Environment* is the operational layer containing the runtime, model context, and proposed action; the *Attesting Environment* is a cryptographically isolated TEE (Intel TDX, AMD SEV-SNP, NVIDIA Confidential Computing) [19, 20] that measures the policy-engine binaries, verifies the policy-bundle signature, evaluates the snapshot, and signs the proof;

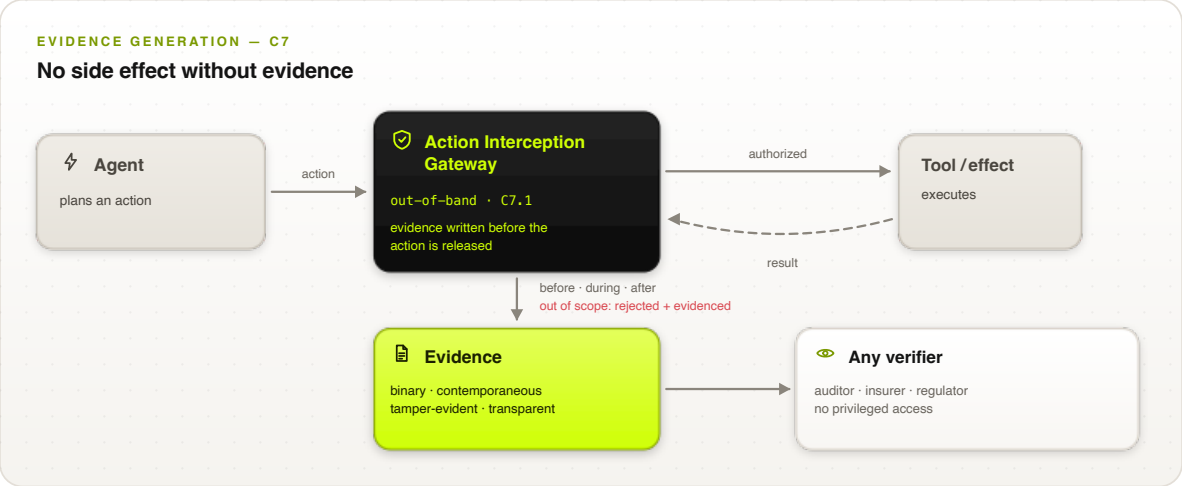

Figure 3: Evidence generation at the action boundary. The out-of-band Action Interception Gateway mediates every tool call, writes evidence before the action is released, and emits records any verifier can check without privileged access; out-of-scope actions are rejected and evidenced.

Evidence is an Entity Attestation Token (CWT/JWT) carrying measurements, claims, step index, verdict, and platform signatures [7, 23]; a *Verifier* compares Evidence against reference values to produce an Attestation Result; and *Relying Parties* (API gateways, downstream services) require valid results before executing tool requests. Sub-agent delegation chains are governed by composite multi-verifier topologies [24].

5.3 Merkle-chain execution history

Each intercepted step t generates a leaf

$$L_t = H(L_{t-1} \,\|\, H(\sigma_t^{\text{snap}}) \,\|\, d_t)\,, \qquad L_0 = H(\text{seed}_{\text{init}}), \tag{1}$$

where σ_t^{snap} is the canonical snapshot, d_t the verdict, and H a collision-resistant hash. The Attesting Environment signs each leaf with an enclave-bound key; per-source monotonic sequence indices make *omission* as detectable as alteration. The accumulated root is periodically anchored to a public ledger or distributed trust root (asynchronously, off the hot path), preventing selective editing, reordering, or suppression of history [8].

5.4 Canonical evidence schema

Every intercepted step yields one Evidence token: an Entity Attestation Token [7] carrying the Proof-of-Control claim set alongside the hardware attestation submodule. Listing 1 shows a representative token (JWT rendering; the CBOR/CWT encoding is byte-equivalent in claim structure), produced at the pre-call tool-invocation hook for a payment API request.

Table 1 defines each claim. Three design decisions matter. First, *payloads never leave the boundary*: the token carries `canonical_snapshot_hash`, a commitment to the full evaluated state, rather than the state itself—so a third party can later confirm exactly what was evaluated (by hashing a disclosed snapshot) without the evidence store ever holding raw data (the C2 privacy requirements). Second, *history is load-bearing*: `step_index` and `merkle_root` bind this token to every token before it, which is what makes silent omission of an inconvenient step as detectable as alteration. Third, *the policy is pinned*: `policy_bundle_hash` identifies the exact rule set that

```
{
  "iss": "https://verifier.trusted-attestation.org",
  "iat": 1773532800,
  "nonce": "0x8f93e2a110c49b",
  "eat_profile": "https://standards.org/poc/v1",
  "poc_claims": {
    "agent_id": "did:web:enterprise.com:agents:finance-agent-04",
    "initiating_user": "user:auth0|84f12a9e",
    "agbom_digest": "sha256:7f8a...5f6a",
    "interception_point": "PRE_CALL_TOOL_INVOCATION",
    "step_index": 3,
    "merkle_root": "0x2c3d...2c3d",
    "policy_bundle_hash": "sha256:e3b0...b855",
    "target_resource": "api.bank.com/v1/payments",
    "canonical_snapshot_hash": "sha256:a4b3...a1b2",
    "verdict": "ALLOW"
  },
  "submods": {
    "hardware_attestation": {
      "platform": "INTEL_TDX",
      "mr_config": "0x1a2b3c4d5e6f...",
      "attestation_signature": "0x30440220..."
    }
  }
}
```


Claim	Type	Meaning
`iss` / `iat`	EAT	Token issuer and issuance timestamp.
`nonce`	EAT	Relying-party challenge; proves freshness and prevents replay of an old token for a new action.
`eat_profile`	EAT	The Proof-of-Control EAT profile (and version) this token conforms to.
`agent_id`	PoC	Decentralized identifier of the agent instance (C5); resolvable to its public keys.
`initiating_user`	PoC	The principal on whose authority the action runs—the human-to-agent binding (C5.1).
`agbom_digest`	PoC	Digest of the Agent Bill of Materials: the build, tools, and model manifest the agent runs from (C1).
`interception_point`	PoC	Which lifecycle hook produced this token (one of the eight in §5).
`step_index`	PoC	Monotonic position in the execution path; gaps expose omitted steps.
`merkle_root`	PoC	Accumulated root of the signed execution chain up to and including this step.
`policy_bundle_hash`	PoC	Digest of the signed Rego bundle that was evaluated—pins the exact policy in force.
`target_resource`	PoC	The resource the proposed action addressed (tool endpoint, table, device).
`canonical_snapshot_hash`	PoC	Commitment to the full canonical state snapshot that was evaluated; discloses nothing by itself.
`verdict`	PoC	The binding decision: ALLOW, DENY, MODIFY, or ESCALATE.
`platform` / `mr_config`	HW	TEE platform and enclave measurement; compared against published reference values to prove the authorized policy engine ran unmodified.
`attestation_signature`	HW	Hardware-rooted signature over the token by the enclave-bound key.

Table 1: The Evidence claim set. "EAT" rows are standard Entity Attestation Token claims [7]; "PoC" rows are the Proof-of-Control profile; "HW" rows form the hardware-attestation submodule.

produced the verdict, so "which policy was in force?" is a claim to check, not a change-log to reconstruct.

How a verifier consumes the token. Verification is a fixed, mechanical sequence: (1) validate `attestation_signature` against the platform's attestation roots (composed with independent anchoring per the Tier-3 rule of Section 4.2); (2) compare `mr_config` to the published golden measurement of the policy engine; (3) check `policy_bundle_hash` against the signed bundle registry; (4) confirm `nonce` freshness; (5) confirm `step_index` continuity and `merkle_root` consistency with previously seen tokens for this `agent_id`; and only then (6) accept `verdict` as evidence that the control held for this action. Every step uses published materials—no privileged access to the operator's infrastructure is required, which is precisely what places this evidence above the binary threshold.

Attribute	**System prompts**	**Static RBAC**	**App guardrails**	**Proof-of-Control**
Enforcement	Probabilistic alignment	Binary API gating	Application code	Hardware-isolated policy evaluation
Context awareness	Context-window scope	None (context-blind)	Partial (local state)	Complete (snapshot + path)
Tamper resistance	Negligible (injection-prone)	Moderate (credential-reliant)	Low (bypassable)	Cryptographic (TEE-backed)
Audit verification	Unauthenticated text	Operator-mutable logs	Operator-mutable logs	Machine-checkable Merkle proofs
Operator trust	Total	Total	Total	Zero-trust (hardware-anchored)

Table 2: Governance mechanisms compared. Only Proof-of-Control clears the binary threshold of Section 4.2.

6 Formal Model

Let $\mathcal{A}$ be the space of agent actions, $\mathcal{I}$ the space of cryptographically verifiable agent identities, and $\mathcal{S}$ the space of host state snapshots. The executed steps up to time t form the partial path $p_t = \langle a_0, a_1, \ldots, a_{t-1} \rangle \in \mathcal{A}^*$. A compliance policy is a deterministic evaluation function

$$\Pi : \mathcal{A} \times \mathcal{I} \times \mathcal{S} \times \mathcal{A}^* \longrightarrow \mathcal{D}, \qquad \mathcal{D} = \{\textsc{allow},\ \textsc{deny},\ \textsc{modify}(a'),\ \textsc{escalate}(h)\}, \tag{2}$$

where $a' \in \mathcal{A}$ is a sanitized payload and h a human-oversight routing profile. Proof-of-Control requires Π to execute inside an isolated enclave, producing per-step proof

$$\pi_t = \mathrm{Sign}_{k_{\mathrm{AE}}}(H(\sigma_t^{\mathrm{snap}}) \,\|\, d_t \,\|\, t \,\|\, L_t)\,, \tag{3}$$

with k_{AE} accessible exclusively to the Attesting Environment. The path argument makes the policy *path-aware*: Π evaluates the proposed action against context carried from upstream invocations, so composition attacks that are invisible to per-action evaluation are within the decision boundary [16]. A deliberate *determinism boundary* limits claims: Proof-of-Control verifies deterministic facts of execution (which model ran, on what input, under which authority, producing which effect); it does not represent output correctness, model reasoning, fairness, or future behavior as verified. Verification is performed, not validation [33].

7 Security Analysis

Proof-of-Control assumes a zero-trust model in which the host OS, network, and operator may be compromised. Representative threats and mitigations:

- **Compromised host operator** (suppressing enforcement, editing logs, issuing unauthorized calls): policy logic and signing keys are enclave-isolated; altering the policy manifest changes the enclave measurement, causing verifiers to reject evidence [19].
- **Prompt injection**: hooks intercept proposed calls before dispatch; the enclave evaluates deterministic rules independently of LLM reasoning, denying unpermitted actions inline [5].
- **Log alteration and omission**: Merkle chaining makes deletion or reordering break the signed root; monotonic sequencing makes silent omission detectable [8].
- **Cascading sub-agent exploitation**: delegation hooks enforce constraint inheritance; sub-agents must produce evidence verified through multi-verifier topologies before actions execute [24].
- **Skill-composition risk**: path-aware authorization (C4) gates capability flow and trust transfer that artifact-level vetting cannot see [16].
- **Monitoring blind spots**: trajectory validators must be evaluated for structured-trace parsing competence, not only natural-language safety performance [25].

The full normative threat model enumerates 32 threats drawn from MITRE ATLAS [38], NIST AI 100-2, and the OWASP catalogues, grading per-threat coverage and stating explicit out-of-scope boundaries; four rows are deliberately marked *not addressed* (e.g., output correctness), which is what keeps the claim honest.

8 Evaluation

8.1 Regulatory coverage: a reproducible mapping

Following the coding methodology of HAARF [9], every PoC requirement was coded against eight frameworks as Exact Match (equivalent clause in scope and intent), Partial Match (same topic without operator-independent evidence, or at lesser depth), or No Match. Coverage = (EM + PM)/111. Coding is single-coder seed data at section granularity, published with per-row rationales and clause attributions for working-group validation; the computation is reproducible from the public coding sheet. Figure 4 and Table 3 report the results.

Two patterns are consistent. First, the partial-match band is wide and the exact-match band thin: NIST AI RMF attains the highest coverage with zero exact matches, because governance frameworks ask for nearly every control PoC verifies while never requiring mechanism-generated evidence. Second, the no-match gap concentrates in the evidence-property, tier-grading, and disclosure chapters: no coded framework grades evidence by how independently it can be verified, requires trust-assumption disclosure, or reaches self-enforcing execution. The gap is, by design, the standard's reason to exist.

8.2 Performance budget

Because agent workflows involve synchronous tool invocations, governance must operate at machine speed. Proof-of-Control targets end-to-end overhead below **15 ms per intercepted step**, achieved

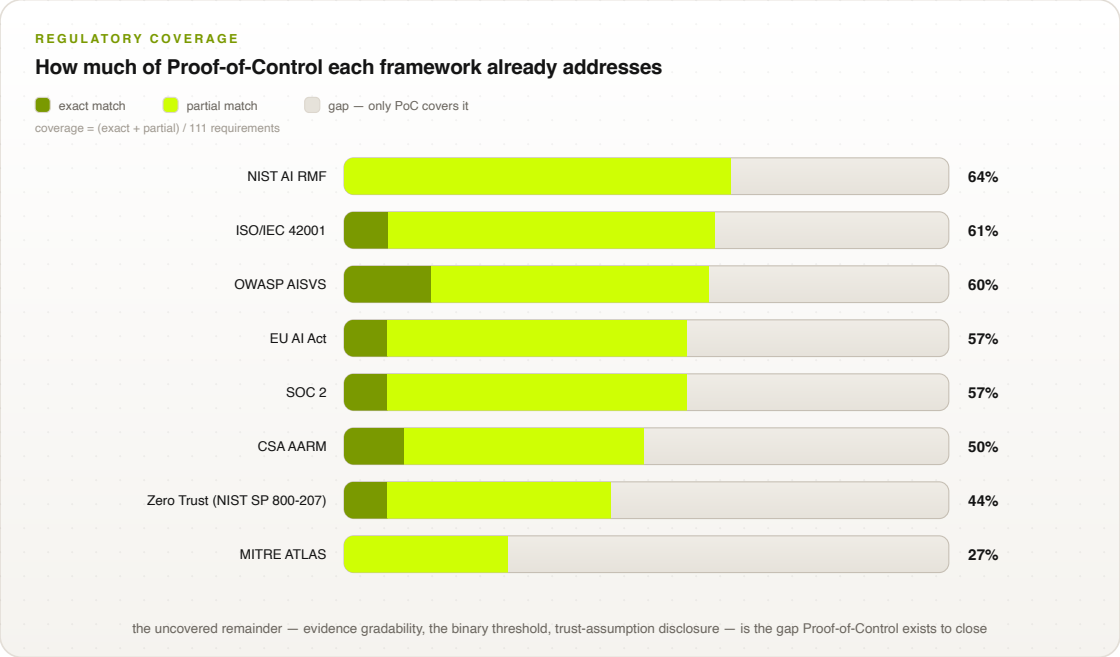

Figure 4: Coverage of the 111 Proof-of-Control requirements by eight external frameworks (draft seed coding). Dark lime: exact matches; bright lime: partial matches; the remaining track is the gap only Proof-of-Control covers.

by (i) policy engines compiled to native WebAssembly/Rust modules executing in enclave memory, eliminating IPC overhead; (ii) hardware-accelerated Ed25519/ECDSA P-256 signatures; and (iii) asynchronous anchoring—leaf generation and signing are inline, while publishing Merkle roots to external ledgers runs in the background. The verification-bandwidth economics motivating this budget are structural: as execution cost falls toward zero, verification becomes the scarce factor [4], so checking compliance must be cheap, binary, and mechanical.

Framework	Exact	Partial	None	Coverage
NIST AI RMF [32]	0	71	40	64%
ISO/IEC 42001 [33]	8	60	43	61%
OWASP AISVS [37]	16	51	44	60%
EU AI Act [36]	8	55	48	57%
SOC 2 [35]	8	55	48	57%
CSA AARM [21]	11	44	56	50%
Zero Trust (SP 800-207) [17]	8	41	62	44%
MITRE ATLAS [38]	0	30	81	27%

Table 3: Requirement-level coverage results. The exact-match band is thin everywhere: existing frameworks require the *controls* but almost never operator-independent *evidence* that they held—the gap between Tier 2 and Tier 3.

9 Discussion

9.1 Standards integration

ISO/IEC JTC 1/SC 42: Proof-of-Control provides a technical realization for active SC 42 workstreams [10, 11, 43]—a technical annex for auditable runtime controls under ISO/IEC 42001 Clauses 8–9 (supplying cryptographically verifiable compliance evidence during certification audits) [33], operationalization of ISO/IEC 23894 risk treatment as executable, hardware-verified policy bundles [34], and an SC 42/SC 27 bridge coupling AI oversight with confidential-computing standards [44]. *IETF RATS:* an Internet-Draft establishing an AI-agent EAT profile [7, 23]—standardizing claims for lifecycle hook types, policy-bundle hashes, Agent Bill of Materials (AgBOM) digests, and step indices—and an extension of multi-verifier attestation to sub-agent delegation chains [24]. *ACS:* Proof-of-Control supplies the missing cryptographic layer for ACS [13, 14], anchoring its decision engine in TEEs and signing decision tokens. *NIST AI 600-1:* Proof-of-Control provides concrete implementations for the generative-AI profile's measurement controls [15, 45, 46]—enforcing that deployment configurations cannot bypass security controls and satisfying continuous security verification with attestation tokens for every tool call and state transition.

9.2 Implementation roadmap

Figure 5 summarizes the path to ratification and adoption. **Phase 1 (months 1–6): open specification and reference core**—core schema extensions on the ACS repository, open-source Rust reference enclave policy engines, and the IETF Internet-Draft for the AI-agent EAT profile. **Phase 2 (months 7–12): multi-vendor interoperability and SDO balloting**—SC 42 working groups, conformance testing across Python, Node.js, and Rust agent frameworks, and telemetry integration with OpenTelemetry and OCSF. **Phase 3 (months 13–24): ratification and deployment**—normative annexes in ISO/IEC 42001 and 23894, RFC publication, and deployments across financial, healthcare, and public-sector workloads. On the adopter side, the same three phases carry an organization from foundational key, signing, and logging infrastructure through expanded proof coverage to continuously monitored operation—each phase reaching readiness for one conformance stage.

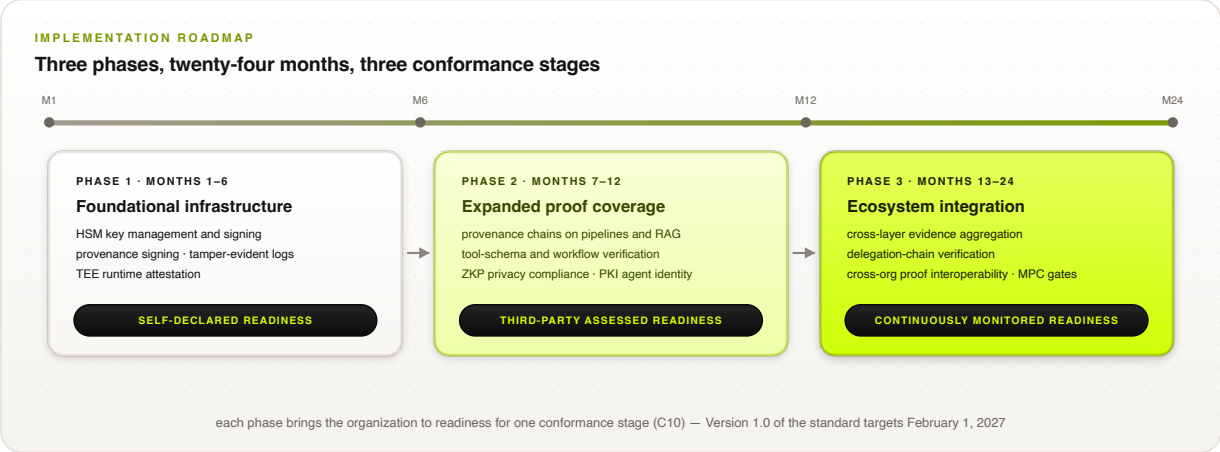

Figure 5: Implementation roadmap. Each phase brings an adopting organization to readiness for one conformance stage; on the standards track, Version 1.0 targets February 1, 2027.

9.3 Limitations

Four limitations are stated deliberately. (1) The determinism boundary: PoC verifies execution facts, not output correctness, fairness, or intent; validation remains a human responsibility. (2) The binary threshold is under dedicated cryptographic and complexity-theoretic review, including impossibility results on what agentic verification can achieve efficiently. (3) The regulatory mapping is a single-coder seed pending multi-coder validation and inter-coder agreement statistics. (4) Tier-4 self-enforcement creates an availability dependency (the system halts when proofs cannot be produced) that deployments must assess and disclose. Open working-group questions—identity/authorization ownership, verifiable-but-unlinkable identity, evidence continuity across jurisdictions as a possible fifth evidence property, and continuous-monitoring cadence—are tracked publicly in the specification's open-issues register.

10 Conclusion

The claims-based world, in which relying parties accept an operator's word for what an autonomous system did, does not scale to agents no one can watch. Proof-of-Control replaces attestation with mechanism-generated, hardware-anchored, independently checkable evidence: six domains of verifiable facts, a binary threshold that makes verifiability procurable, an architecture that intercepts every consequential action, and a coverage evaluation demonstrating that no existing framework occupies this layer. The specification, audit checklist, coding sheets, and reference tooling are developed openly under the Advanced AI Society and are freely licensed for implementation.

Acknowledgments

This work is developed within the Proof-of-Control Initiative of the Advanced AI Society. The authors thank the domain working groups and contributors whose input shaped the draft standard, including Bob Blessing-Hartley, Advait Patel, David Thomson, Drummond Reed, and Hart Montgomery (Linux Foundation Decentralized Trust), and the Cloud Security Alliance MAESTRO and AARM communities. The Proof-of-Control Lab is being established as a community lab at Linux Foundation Decentralized Trust.

References

[1] Palo Alto Networks. A complete guide to agentic AI governance. https://www.paloaltonetworks.com/cyberpedia/what-is-agentic-ai-governance, 2026.

[2] Airia. What is AI agent governance? A framework for keeping agents in check. https://airia.com/blog/what-is-ai-agent-governance-a-framework-for-keeping-agents-in-check/, 2026.

[3] Advanced AI Society. Open verification: the proof-of-control standard for agents, working draft v0.1.4. https://advancedaisociety.org/, 2026. Specification repository: requirement chapters C1–C10, appendices, audit checklist, and regulatory coding sheets.

[4] Christian Catalini, Xiang Hui, and Jane Wu. Some simple economics of AGI. *arXiv preprint arXiv:2602.20946*, 2026. https://arxiv.org/abs/2602.20946.

[5] Runtime Governance Working Group. Runtime governance for AI agents: Policies on paths. *arXiv preprint arXiv:2603.16586*, 2026. https://arxiv.org/html/2603.16586v1 [verify author list].

[6] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote ATtestation procedureS (RATS) architecture. RFC 9334, Internet Engineering Task Force, 2023.

[7] Laurence Lundblade, Giridhar Mandyam, Jeremy O'Donoghue, and Carl Wallace. The entity attestation token (EAT). RFC 9711, Internet Engineering Task Force, 2025.

[8] r/AI_Agents contributors. Signed Merkle-chain audit log for AI agent tool calls: Offline verifier, hybrid PQ. https://www.reddit.com/r/AI_Agents/comments/1v2nv08/signed_merklechain_audit_log_for_ai_agent_tool/, 2026. Community reference implementation discussion.

[9] Task Force for AI Agents in Healthcare. HAARF: A unified, lifecycle-centric regulatory framework for autonomous AI agents in medical environments. https://github.com/Task-force-for-AI-agents-in-Healthcare/haarf, 2026. Coverage-mapping methodology (EM/PM/NM rubric, coding sheets, reproducible coverage computation).

[10] ISO/IEC JTC 1. SC 42: Artificial intelligence — subcommittee overview. https://jtc1info.org/sd-2-history/jtc1-subcommittees/sc-42/, 2026.

[11] Nemko Digital. ISO/IEC JTC 1/SC 42: Leading AI standards for 2025. https://digital.nemko.com/standards/iso-iec-jtc-1sc-42, 2025.

[12] IETF RATS Working Group. Remote ATtestation procedureS (rats): Charter and documents. https://datatracker.ietf.org/group/rats/about/, 2026.

[13] Agent Control Standard. Agent control standard launches open framework for runtime governance of AI agents. https://www.businesswire.com/news/home/20260527326259/en/Agent-Control-Standard-Launches-Open-Framework-for-Runtime-Governance-of-AI-Agents, 2026.

[14] Microsoft Command Line. Agent control specification: Portable runtime governance for AI agents. https://commandline.microsoft.com/agent-control-specification-runtime-governance/, 2026.

[15] National Institute of Standards and Technology. Artificial intelligence risk management framework: Generative artificial intelligence profile. NIST AI 600-1, NIST, 2024.

[16] et al. Xie. SCR-Bench: Measuring skill composition risk in sandboxed agent execution, 2026. [verify: full author list, venue, and identifier pending confirmation against the primary source].

[17] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. Zero trust architecture. NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.

[18] NETBankAudit. Generative AI controls and risk assessments for financial institutions. https://www.netbankaudit.com/resources/generative-ai-controls-risk-assessments, 2026.

[19] Red Hat. Attestation in confidential computing. https://www.redhat.com/en/blog/attestation-confidential-computing, 2026.

[20] EQTY Lab. Verifiable compute: AI integrity suite. https://www.eqtylab.io/blog/verifiable-compute-press-release, 2026.

[21] Cloud Security Alliance. Autonomous action runtime management (AARM). https://cloudsecurityalliance.org/, 2026. Contributed by Vanta.

[22] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote attestation procedures architecture. Internet-Draft draft-ietf-rats-architecture-07, Internet Engineering Task Force, 2021.

[23] Ned Smith. TCG attestation framework and IETF remote attestation procedures. https://trustedcomputinggroup.org/wp-content/uploads/2_N.Smith_TCG-JRF02292024.pdf, 2024. Trusted Computing Group.

[24] IETF RATS Working Group. Remote attestation with multiple verifiers. Internet-Draft draft-ietf-rats-multi-verifier-00, Internet Engineering Task Force, 2026.

[25] et al. Chen. TraceSafe-Bench: Evaluating guardrail efficacy on corrupted multi-step tool-calling trajectories, 2026. [verify: full author list, venue, and identifier pending confirmation against the primary source].

[26] et al. Bandara. AI trust OS: Telemetry-first compliance for enterprise AI governance, 2026. [verify: full author list, venue, and identifier pending confirmation against the primary source].

[27] MindXO. Enterprise AI key risk indicator (KRI) taxonomy, 2026. [verify: publication identifier pending confirmation].

[28] World Wide Web Consortium. Decentralized identifiers (DIDs) v1.0. https://www.w3.org/TR/did-core/, 2022.

[29] World Wide Web Consortium. Verifiable credentials data model v2.0. https://www.w3.org/TR/vc-data-model-2.0/, 2025.

[30] Web 7.0 Foundation. Verifiable trust circles: N-party trust constructs over W3C verifiable credentials data model 2.0, 2026. [verify: specification identifier pending confirmation].

[31] Catena-X Automotive Network. AI service KIT: Identity-anchored provenance for cross-organizational ML pipelines. https://catena-x.net/, 2026. [verify: KIT release identifier].

[32] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). NIST AI 100-1, NIST, 2023.

[33] International Organization for Standardization. ISO/IEC 42001:2023 — information technology — artificial intelligence — management system. https://www.iso.org/standard/81230.html, 2023.

[34] International Organization for Standardization. ISO/IEC 23894:2023 — artificial intelligence — guidance on risk management. https://www.iso.org/standard/77304.html, 2023.

[35] AICPA. Trust services criteria (2017, with 2022 revised points of focus). https://www.aicpa-cima.com/resources/download/trust-services-criteria, 2022.

[36] European Union. Regulation (EU) 2024/1689 — artificial intelligence act. https://eur-lex.europa.eu/eli/reg/2024/1689/oj, 2024. Official Journal of the European Union, 12 July 2024.

[37] OWASP Foundation. Artificial intelligence security verification standard (AISVS), v1.0. https://github.com/OWASP/AISVS, 2026.

[38] MITRE Corporation. MITRE ATLAS: Adversarial threat landscape for artificial-intelligence systems. https://atlas.mitre.org/, 2026.

[39] Scott Bradner. Key words for use in RFCs to indicate requirement levels. RFC 2119, Internet Engineering Task Force, 1997.

[40] OpenSSF. SLSA: Supply-chain levels for software artifacts. https://slsa.dev/, 2026.

[41] Coalition for Content Provenance and Authenticity. C2PA technical specification. https://c2pa.org/, 2026.

[42] Ken Huang. MAESTRO framework: Navigating risks in multi-agent AI systems. https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro, 2025. Cloud Security Alliance.

[43] LexAI Europe. ISO/IEC standards on AI. https://www.lexai-europe.com/1901/standardisation/iso-iec-ai, 2025.

[44] International Electrotechnical Commission. Coordinated standardization for AI and data security in the era of generative AI: Strategic proposals. https://iec.ch/system/files/2025-10/wsd_2025_combinedpdf.pdf, 2025.

[45] CipherNorth. NIST AI 600-1 and AI RMF: Managing risk in generative AI. https://www.ciphernorth.com/blog/nist-ai-risk-management-framework-rmf, 2026.

[46] Modulos. Implement NIST AI risk management framework. https://www.modulos.ai/nist-ai-rmf/, 2026.